

Gift / Points Sharing — Clean API Procedures Guide (for C# API)

This document clearly explains the stored procedures required for your C# API, with their purposes, inputs/outputs, validations, and when to call them. It is reformatted for readability.

1) Quick Overview of Procedures

- `sp_send_share_request(owner_id, requester_id, requested_percent)` — Recipient sends or updates a pending share request to an owner.
- `sp_respond_share_request(request_id, owner_id, accept)` — Owner accepts or rejects a request; accepted requests update allocations.
- `sp_share_owner_amount(owner_id, amount_rs, note)` — Distributes actual money/points from owner to recipients per active allocations.
- `fn_total_percent_for_owner(owner_id)` — Read-only function to compute the total % allocated by an owner.

2) `sp_send_share_request(owner_id, requester_id, requested_percent)`

Purpose: Insert a new pending request or update the percentage of an existing pending request from a recipient to an owner.

Key Validations:

- Owner and requester must not be the same.
 - Requested percent must be between 0 and 100.
 - Both owner and requester must exist in users table.
- Outcome: Creates or updates a row in share_requests with status='pending'.

Example:

```
CALL sp_send_share_request(20, 23, 20.00);
```

3) `sp_respond_share_request(request_id, owner_id, accept)`

Purpose: Owner accepts or rejects a specific pending request.

On Accept:

- Computes total allocations including new %.
- Enforces total $\leq 100\%$ (rollback if exceeded).
- Upserts into `share_allocations` and links to `request_id`.
- Marks `share_requests` row as accepted.

On Reject: Marks `share_requests` row as rejected.

Example:

```
CALL sp_respond_share_request(3, 20, TRUE);  -- accept
CALL sp_respond_share_request(4, 20, FALSE); -- reject
```

4) `sp_share_owner_amount(owner_id, amount_rs, note)`

Purpose: Execute the actual distribution transaction from owner to recipients based on active allocations.

Behavior:

- Validates amount > 0.
- Locks owner wallet row; checks balance.
- Deducts owner's balance.
- Credits each recipient by their share (rounded to 2 decimals).
- Writes transfer ledger entries with snapshot of %.
- Refunds rounding residue to owner.
- All within a single transaction.

Example:

```
CALL sp_share_owner_amount(20, 200.00, 'Monthly share');
```

5) fn_total_percent_for_owner(owner_id)

Purpose: Compute the sum of active allocation percentages for an owner (read-only helper).

Usage: Used by UI or API to display total allocations before allowing new ones.

Example:

```
SELECT fn_total_percent_for_owner(20) AS total_alloc;
```

6) Suggested API Endpoints (C#)

- POST /share-requests → CALL sp_send_share_request
- POST /share-requests/{id}/respond → CALL sp_respond_share_request
- POST /wallets/{ownerId}/share → CALL sp_share_owner_amount
- GET /owners/{ownerId}/allocations/total → SELECT fn_total_percent_for_owner
- GET /owners/{ownerId}/allocations → SELECT * FROM share_allocations WHERE owner_id=? AND is_active=1
- GET /owners/{ownerId}/transfers → SELECT * FROM transfers WHERE owner_id=? ORDER BY created_at DESC

7) Error Handling & Mapping

Procedures SIGNAL SQLSTATE '45000' with messages. Map as:

400 Bad Request -> invalid inputs (percent range, same owner & requester, amount <=0)

404 Not Found -> owner/requester/request not found

409 Conflict -> exceeds 100% allocation or insufficient owner balance

200 OK / 201 Created -> success

8) Important Fix

In `sp_share_owner_amount`, replace references to 'wallets' with 'user_wallets'. Add `UNIQUE(user_id)` to enforce one wallet per user.

```
ALTER TABLE user_wallets ADD CONSTRAINT ux_wallet_user UNIQUE (user_id);
```